

Topic 1[Method/Function Overloading]

Problem Statement: Define a class Test where overload a method Sum() to sum numbers sent from main() function.

Solution:

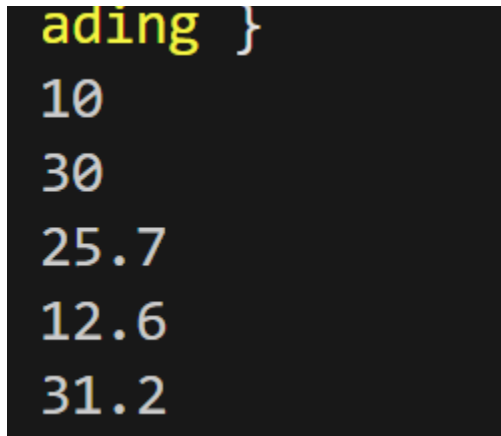
```
#include<bits/stdc++.h>
using namespace std;
class Test
{
    public:
    int Sum(int t)
    {
        return t;
    }
    int Sum(int a, int b)
    {
        return a+b;
    }
    double Sum(double a, int b)
    {
        return a+b;
    }
    double Sum(int a, double b)
    {
        return a+b;
    }
    double Sum(double a, double b)
    {
        return a+b;
    }
};
int main()
```

```

{
    Test t;
    cout << t.Sum(10) << endl;
    cout << t.Sum(10,20) << endl;
    cout << t.Sum(5.7,20) << endl;
    cout << t.Sum(10,2.6) << endl;
    cout << t.Sum(10.5,20.7) << endl;
    return 0;
}

```

Output:



```

ading }
10
30
25.7
12.6
31.2

```

Topic 2 [Binary Operator Overloading]: Suppose in a AC circuit, there are 3 impedances $z_1=3+j4$, $z_2=4-j3$ and $z_3=j6$ are connected in parallel. Now find the current in the circuit if input voltage is $100+j50$. Implement operator overloading concept for your calculation. Use class Circuit and initialize the impedance values (real & img) by a constructor.

Solution:

```

#include <iostream>
using namespace std;

class Circuit {
private:

```

```

    double real;
    double img;

public:
    Circuit(double r = 0, double i = 0) {
        real = r;
        img = i;
    }
    Circuit operator+(Circuit c) {
        return Circuit(real + c.real, img + c.img);
    }
    Circuit operator/(Circuit c) {
        double denom = c.real * c.real + c.img * c.img;
        double r = (real * c.real + img * c.img) / denom;
        double i = (img * c.real - real * c.img) / denom;
        return Circuit(r, i);
    }
    void display() {
        if (img >= 0)
            cout << real << " + j" << img << endl;
        else
            cout << real << " - j" << -img << endl;
    }
};

int main() {
    Circuit z1(3, 4);
    Circuit z2(4, -3);
    Circuit z3(0, 6);

```

```

Circuit V(100, 50);
Circuit one(1, 0);
Circuit y1 = one / z1;
Circuit y2 = one / z2;
Circuit y3 = one / z3;

Circuit Yeq = y1 + y2 +
y3;

Circuit Zeq = one / Yeq;

Circuit I = V / Zeq;

cout << "Equivalent Impedance Z = ";
Zeq.display();

cout << "Circuit Current I = ";
I.display();

return 0;
}

```

```

PS D:\Satu\1-2\CSE 1204\week 4> cd "d:\Satu\1
g++ binaryoprtroverloading.cpp -o binaryoprtro
yoprtroverloading }
Equivalent Impedance Z = 2.31193 + j1.70642
Circuit Current I = 38.3333 - j6.66667

```

Topic 3 [Unary Operator Overload]

Problem statement: You need to write a program for remote controller of Air-conditioner. You need the control two buttons like temperature up and temperature down using the concept of unary operator overloading. The main() function would be look like:

Solution:

```
#include <iostream>
```

```

using namespace std;
class AirConditioner
{
    int temperature;
public:
    AirConditioner(int t)
    {
        temperature = t;
    }
    void operator++()
    {
        temperature++;
    }
    void operator--()
    {
        temperature--;
    }
    void display()
    {
        cout << "Current Temperature: " << temperature << endl;
    }
};

int main()
{
    AirConditioner ac(24);
    ac.display();
    ++ac;
    ac.display();
    --ac;
    ac.display();
    return 0;
}

```

```

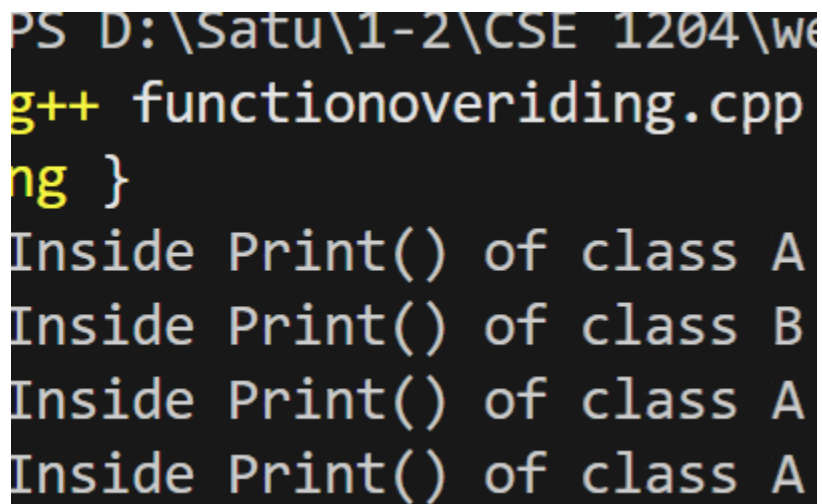
PS D:\Satu\1-2\CSE 1204\week 4> c
g++ unary.cpp -o unary } ; if ($?
Current Temperature: 24
Current Temperature: 25
Current Temperature: 24
PS D:\Satu\1-2\CSE 1204\week 4>

```

Problem statement: Write a class A with a method Print() and a derived class B with method Print() overloaded. Now observe the output when following statements are written in the main() function-

Solution:

```
#include <iostream>
using namespace std;
class A{
public:
    void Print(){
        cout << "Inside Print() of class A" << endl;
    }
};
class B : public A{
public:
    void Print(){
        cout << "Inside Print() of class B" << endl;
    }
};
int main()
{
    A a;
    a.Print();
    B b;
    b.Print();
    A a2;
    A *p1;
    p1 = &a2;
    p1->Print();
    B b2;
    A *p2;
    p2 = &b2;
    p2->Print();
    return 0;
}
```

A screenshot of a C++ program's output. The code defines two classes, A and B, both with a Print() method. Class B inherits from class A. In the main function, four Print() calls are made: first on a direct instance of A, then on a direct instance of B, then on a pointer to an instance of A, and finally on a pointer to an instance of B. The output shows that the first and third calls print "Inside Print() of class A", while the second and fourth calls print "Inside Print() of class B", demonstrating that the derived class's method is called when the object is of type B, even if it is accessed through a pointer of type A.

```
PS D:\Satu\1-2\CSE 1204\we
g++ functionoverriding.cpp
ng }
Inside Print() of class A
Inside Print() of class B
Inside Print() of class A
Inside Print() of class A
```

```
}
```

Topic 5 [Friend Function]

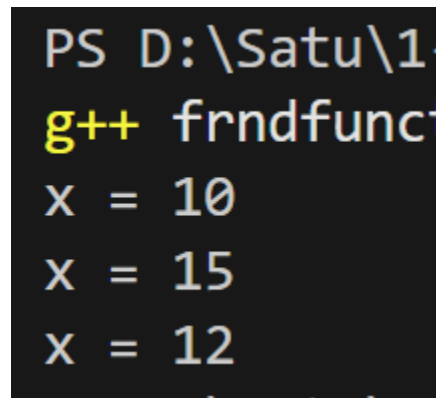
Problem Statement: Using the following class, write three friend functions

- i) Add(): Assign value to the data member x
- ii) IncX() : Increase the value of x by m
- iii) DecX() : Decrease the value of x by n

Solution:

```
#include <iostream>
using namespace std;
class Sample
{
    int x;
public:
    Sample()
    {
        x = 0;
    }
    friend void Add(Sample &s, int value);
    friend void IncX(Sample &s, int m);
    friend void DecX(Sample &s, int n);
    void display()
    {
        cout << "Value of x: " << x << endl;
    }
};

void Add(Sample &s, int value)
{
    s.x = value;
```



```
PS D:\Satu\1.
g++ frndfunc
X = 10
X = 15
X = 12
```

```

}

void IncX(Sample &s, int m)
{
    s.x = s.x + m;
}

void DecX(Sample &s, int n)
{
    s.x = s.x - n;
}

int main()
{
    Sample obj;
    Add(obj, 10);
    obj.display();
    IncX(obj, 5);
    obj.display();
    DecX(obj, 3);
    obj.display();
    return 0;
}

```

Topic 6[Pure Virtual Function]

Problem statement: Modify the class defined in Topic 3 executes the following

```

#include <iostream>
using namespace std;

class A{
public:

```



```
        virtual void Print() = 0;
};

class B : public A{
public:
    void Print(){
        cout << "Inside Print() of class B" << endl;
    }
};
```